

hideout

Operator Handbook

Version 1.0 · May 2026 · Confidential

Prepared for	Erik — Partner review
Prepared by	Brice Ikouebe, Hideout Miami
Date	May 2026
App version	Prototype v1.0 — Build SUCCEEDED, 0 errors
Codebase	HideoutApp / BRICE-OS workspace — 36 files, 3,042+ lines
Purpose	Orient Erik on what was built, how every decision was made, and why the app works the way it does.

This document is not a technical spec. It is an operating manual for the relationship the app is designed to support.

What's in this document

1 Executive Summary

Why Hideout built an app — and what it is not

2 The Business Problem

Five relationships the app is designed to capture

3 Product Doctrine

The rules that govern every feature decision

4 App Tour — Six Tabs

What each screen does and why it was built that way

5 The Admin Layer

The hidden operator surface that prevents chaos

6 The Four Revenue Engines

How the app supports each revenue stream

7 What Is Built vs Stubbed

Honest status of every feature

8 Photography Doctrine

Why photos matter and what good looks like

9 Roadmap

Before Sunday · After Sunday · Phase 2 · Phase 3

10 The Governing Question

The single test for every future feature decision

Executive Summary

Software downstream of how the place already works.

Hideout is a neighborhood cafe in Edgewater, Miami. It has a warm, specific identity, a real community, and a growing set of recurring relationships — with walk-in regulars, office accounts, event organizers, and the weekly Sunday gathering known as The Village.

The app is not a growth hack. It is not a customer acquisition tool. It is not Starbucks. It is a relationship capture system — the digital layer that makes existing Hideout relationships easier to repeat, harder to drop, and visible to the operator as structured data instead of memory.

Before the app existed, most of Hideout's recurring value lived in:

- Memory — Brice remembering who ordered what
- Text messages — coordinating cold brew, event inquiries, Sunday lineups
- Manual invoices — Jimmy's gallon cold brew charged after the fact
- Walk-in variability — no mechanism to smooth revenue week to week

The app converts memory into system. It does not invent new demand. It captures, standardizes, and reduces friction on relationships that already exist.

What it is	A relationship capture system — the digital version of dependability over excitement
What it is not	A growth hack, loyalty game, social platform, or Starbucks-style ordering kiosk
Who it serves first	Morning Regulars, Jimmy, nearby offices, event organizers, Sunday DJs
Governing question	Does this make repeat interactions shorter?
Design doctrine	Dependability beats excitement. Software follows reality.
Revenue goal	Survival \$520/day → Stability \$590/day via recurring, predictable relationships

The Business Problem

Walk-in traffic is volatile. A Tuesday morning can be \$200. The following Tuesday can be \$650. The difference is not the food. It is whether relationships have been made predictable.

Hideout needs revenue that is repeatable, predictable, and low-friction to operate. The app exists to convert five specific relationship types into that kind of revenue.

RELATIONSHIP 1 — THE MORNING REGULAR

Who they are	Comes 3-5x per week. Usually the same order. Time-sensitive — before work or on a walk.
Before the app	Waits in line, orders verbally. If Brice is busy, friction. No mechanism to order ahead.
App job	One tap reorder from Today tab. Order ready before they arrive. NFC tap at counter for zero-friction launch.
Why it matters	15 regulars ordering ahead 4x/week at \$11 average = \$2,640/week in predictable revenue.
Key feature	Today tab — Quick Reorder button. Express Order sheet on NFC tap.

RELATIONSHIP 2 — JIMMY (COLD BREW ACCOUNTS)

Jimmy Holmquist is a real customer. He lives in the same building as Hideout (600 NE 36th St, T3). He orders gallon cold brew for events at \$45 per gallon.

Invoice reference	Invoice #10045 — May 7, 2026 — 3 gallons cold brew — \$135 + \$9.45 tax = \$144.45. Invoice was 21 days overdue when the app was being built.
Before the app	Text Brice. Wait. Arrange pickup. Manual invoice. Chase payment.
App job	Jimmy logs in, selects gallons, picks a date 48hrs out, card auto-charged. No invoice chase.
Why Jimmy matters	He proves the model. The order and willingness to pay already exist. The app formalizes existing reality — it does not invent a new behavior.

Revenue type	Recurring B2B — shows up regardless of walk-in volume. \$135+/order, potentially weekly.
Status	Jimmy is seeded as first B2B account in the codebase — ready to onboard when backend is live.

RELATIONSHIP 3 — OFFICE BREAKFAST ACCOUNTS

Hideout is in Edgewater, surrounded by office buildings, co-working spaces, and residential towers. A single recurring office account at \$150/week is \$7,800/year at near-zero marginal effort after setup.

Target accounts	SkyView 22, A Better You, Expansive, Watermarc — all within walking distance
Package	1 toast + 1 drink per person, \$16/person, minimum 5 persons — locked in code
Before the app	Brice has a conversation, quotes verbally, follows up, invoices manually.
App job	Form submission in Catering tab → standing order → Square payment link → auto-recurring.
Why it matters	Five organizations ordering weekly beats fifty random walk-ins. Same floor space, predictable prep, reliable revenue.

RELATIONSHIP 4 — PRIVATE EVENTS (THE KIMBERLY THREAD)

On May 28, 2026, Kimberly Lagunzad from Built In One reached out to inquire about hosting a 50-person networking event on June 27. The conversation — routed through a coordinator — took multiple back-and-forth messages to establish basic pricing terms. That conversation became the spec for the Events system.

The problem	Venue inquiries arrive via text, Square messages, Instagram DM. Every one requires manual negotiation, custom quote, deposit chase.
What emerged	Standardized model: \$200/hour flat rate for private events. 25% deposit to hold. Catering separate from venue rental.
App job	EventRequest form in Catering tab → operator reviews in admin Events Queue → deposit invoice sent.
Pricing locked	\$200/hour flat rate. 25% deposit. Catering is Hideout menu items at bulk pricing.

The copy decision	"Submit a request. We'll confirm fit, availability, and pricing." — never "Book now." Events are operator-reviewed, not auto-booked.
Status	PrivateEventsView and EventRequestSheet are built. Admin Events Queue built. Backend persistence is Phase 2.

RELATIONSHIP 5 — SUNDAY DJs AND THE VILLAGE

What it is	A free weekly gathering. Every Sunday, 11AM–2PM. Free. Always. Music in the room when people arrive. No agenda, no programming, no pressure.
Before the app	DMs to invite DJs. Memory of who played when. No system for managing requests or lineup.
App job	Sunday tab surfaces the current lineup. Play Request form lets DJs submit. Lineup Manager in admin manages slots.
The doctrine constraint	The Village tab has NO RSVP, no tickets, no attendance count, no social layer. The room belongs to the people in it.
Sunday push notification	One push, Sunday morning, 10–10:30AM. Copy: 'The Village. Sunday. 11AM. DJ Marcos. Come as you are.' Nothing more.
Status	Sunday tab built. PlayRequestSheet built. Admin SundayLineupManagerView built.

Product Doctrine

These are not design preferences. They are rules. If a proposed feature violates them, it does not ship. The physical Hideout already has a character. The app must extend that character, not contradict it.

Dependability beats excitement.

No hype, no manufactured urgency, no 'amazing offer' energy. A customer who opens the app three months after downloading should find it exactly as useful as day one — because it now knows them better.

Relationship Compression.

The first interaction requires explanation. Every subsequent interaction should require less. Jimmy's first gallon cold brew order takes 90 seconds. His tenth takes 10 seconds. Any feature that makes repeat interactions longer is wrong for this product.

Pickup-first, always.

Hideout is a supply business. Pickup is the default. Delivery is supported but visually secondary. Square handles delivery fulfillment when it occurs. The app must never drift toward DoorDash mental models.

Software follows reality.

Jimmy was already ordering cold brew. Kimberly was already inquiring about events. The Village was already happening every Sunday. The app captures and standardizes things that already work.

The app is infrastructure, not novelty.

The app earns its place by being quietly useful, reliably accurate, and increasingly invisible. A customer who barely notices they're using it — who just taps and it works — is the success state.

No engagement loops.

No streaks. No achievement badges. No 'you're on a roll.' No re-engagement push notifications. No 'we miss you' messages. No gamification of any kind. Loyalty points are acknowledgment, not manipulation.

The Village is not an event platform.

The Sunday tab has no RSVP, no tickets, no attendance count, no social layer. It shows who is playing and when. It lets DJs submit play requests. Every addition to the Sunday tab must pass the test: does this serve the people in the room, or does it serve an idea of the room?

Events are requests, not bookings.

Private event inquiries are reviewed by the operator before confirmation. "Submit a request. We'll confirm fit, availability, and pricing." — never "Book now."

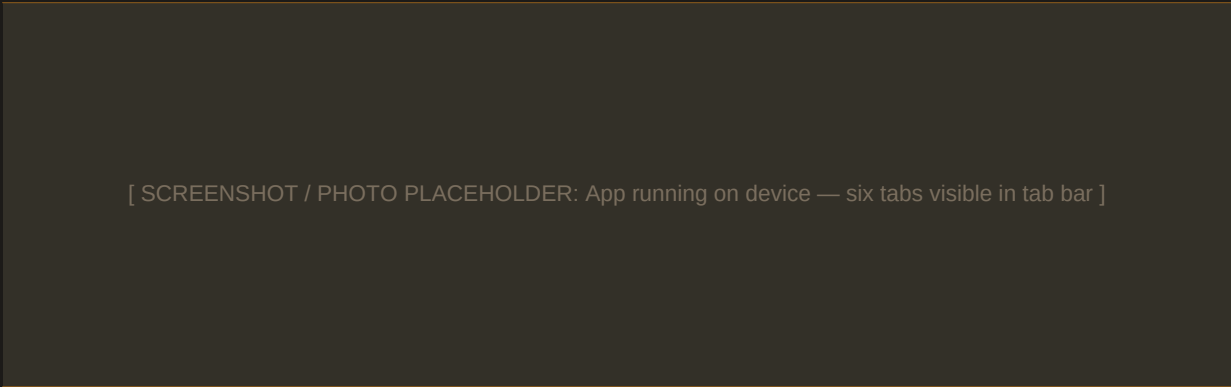
One caramel element per screen.

The only accent color is rationed. Every screen has exactly one caramel element: the primary action. The customer's eye follows caramel. It should always lead to the next right action.

THE FEATURE FILTER Does this make repeat interactions shorter? Not: 'is this useful?' — useful is too broad. Not: 'is this impressive?' — impressive is wrong. Shorter is testable. If a proposed feature expands repeat interactions, it does not ship.

App Tour — Six Tabs

The app has six tabs. Each exists to solve a specific customer problem and create a specific business benefit. None of them are decorative. Every design decision was made to support the behavioral architecture, not to look impressive.



TODAY		The opening screen — center of gravity	
Customer problem		Business benefit	
A regular at 7:30AM has 4 seconds. They need to know: is it open, what's the special, can I reorder in one tap? If those three questions aren't answered immediately, they order at the counter. The app loses.		Quick reorder drives repeat transactions without marketing spend. Village preview surfaces Sunday attendance. Loyalty points snapshot reinforces the relationship without gamification.	
Doctrine: Remembering, repeating, reducing friction.			

ORDER		The full menu — browse and add to cart	
Customer problem		Business benefit	
A customer who wants something new needs a clear, unhurried menu. Category navigation, real food photography, one-tap add. Group ordering is the bridge between individual and office-scale behavior.		Individual orders reduce counter queue friction during busy periods. Group order entry point is the on-ramp to recurring office accounts — someone organizes a team order, eventually sets up a B2B account.	
Doctrine: Software follows reality. Pickup-first.			

REWARDS

Points and loyalty — visible when they matter

Customer problem

Customers check their Square loyalty balance in CashApp or don't use rewards at all. Points need to surface at checkout — the moment they can actually reduce the payment amount.

Business benefit

Rewards displayed at checkout remove the manual 'do you want to use points?' conversation at the counter. Birthday rewards are automated. QR code enables in-store redemption.

Doctrine: No engagement loops. Points are acknowledgment, not manipulation.

CATERING

High-value orders — breakfast, cold brew, events

Customer problem

Office breakfast orders, cold brew by the gallon, and event catering require Brice involved at every step. An office manager wanting 10 boxes on Monday shouldn't need to text on Sunday night.

Business benefit

Catering is where stability revenue lives. The gap between a one-time catering order and a recurring standing account is one good experience. The app makes that transition frictionless.

Doctrine: Relationship Compression. Five organizations weekly beats fifty random walk-ins.

SUNDAY

The Village — lineup and play requests

Customer problem

Village attendees want to know who is playing without checking Instagram or texting. DJs want to submit play requests without a DM chain. Neither experience should feel like an event platform.

Business benefit

The Sunday tab deepens the relationship between Hideout and its creative community without formalizing what should stay informal. A Village regular who uses the app for Sunday info is one step closer to ordering breakfast Thursday.

Doctrine: The Village is not an event platform. Restraint preserved.

ACCOUNT

Profile and preferences — yours to manage

Customer problem

A customer should manage their order history, payment method, notification preferences, and loyalty QR without navigating a complex settings system.

Business benefit

Per-category notification preferences mean customers opt into what they actually want — Sunday reminders, order updates, specials — increasing the signal quality of every push sent.

Doctrine: No engagement loops. Infrastructure, not novelty.

The Admin Layer

The admin layer is the operator's control surface. It is not visible to customers. Accessed via a passphrase-protected 'Staff access' link in the Account tab. For a solo operator, the admin dashboard is where demand is converted from chaos into scheduled, manageable work.

The admin layer exists as a fully built interface in the current app. It becomes fully operational when the backend is built. Until then, it previews the system and allows content management for features that don't require persistence.

Today Content Manager

Set the featured item for the Today tab. Publish today's special. Toggle open/closed status with hours. What Brice opens at 6:45AM before the cafe opens.

Sunday Lineup Manager

Manage upcoming Sunday slots. Move a slot from Open → Held → Confirmed. Set DJ name and vibe note that appear on the Sunday tab. Review and manage play requests from DJs.

Events Queue

Review incoming event requests from the Catering tab. See event details, headcount, date, contact. Approve or decline. Send deposit invoice. Kimberly's June 27 request would appear here.

B2B Account Manager

View all recurring B2B accounts and their standing orders. Jimmy and future office accounts. See upcoming fulfillments, billing status, and modification requests.

Weekly Recurring Calendar

Monday-Sunday view of all B2B fulfillments — who is scheduled, what time, what items, what status. The iPad view a solo operator uses between rushes to see the week ahead.

Broadcast Composer

Send push notifications to specific customer segments. Village opt-ins. Specials subscribers. All customers. 150-character message with preview before send.

Customer Lookup

Find any customer by name, email, or phone. See order history, loyalty balance, account type. Adjust loyalty points manually with a required reason — all admin actions are audit logged.

Cold Brew Availability

Toggle gallon cold brew availability. Set flavors on/off. Set available pickup dates. Maximum 5 gallons per day at current production capacity.

The Four Revenue Engines

Not all revenue is equal. A \$150 recurring office order is structurally different from \$150 in random walk-in transactions — even though the dollar amount is identical. Recurring revenue is predictable, low-friction to repeat, and compounds over time. The app is designed to grow the recurring engines while supporting the walk-in baseline.

Revenue Engine	Current State	App's Role	Potential
Walk-in Regulars (Morning, daily)	Exists — volatile. Depends on foot traffic.	Quick reorder · NFC tap · Scheduled pickup	\$12k-16k/day at stability
B2B / Office (Recurring weekly)	Jimmy: proven. Offices: manual or none.	Standing order · Auto-charge · Admin calendar	\$1k-1.5k/week by account
Private Events (Monthly)	Kimberly: manual. No standardized flow.	EventRequest form · Events Queue · Setup	\$2k-5k/month + Catering add-on
Gallon Cold Brew (B2B recurring)	Jimmy: manual invoice. \$45/gallon proven.	Preorder form · 48hr lead · Auto-charge	\$1k-1.5k/month steady demand

Revenue bands that govern product priority: Survival: \$520/day · Stability: \$590/day · Comfort: \$650/day · Growth: \$750/day The app's job at launch: push from Survival toward Stability through retention and recurring B2B. Discovery — finding new customers — is not the app's job.

What Is Built vs Stubbed

The app builds successfully with 0 errors and runs on device. This table is an honest accounting of what works, what is designed but not yet connected, and what is not yet built. Nothing here is exaggerated in either direction.

Feature	Status	Notes
CUSTOMER-FACING FEATURES		
Today — open status, featured item, quick reorder	BUILT	Stub data — needs backend to be live
Today — NFC/QR source routing	BUILT	URL routing wired — needs NFC sticker at counter
Today — Express Order sheet (counter tap)	BUILT	Appears for returning users on NFC launch
Today — Quick Order chips	BUILT	Functional — adds to cart immediately
Order tab — full menu catalog (17 items)	BUILT	Real prices, correct modifiers from approved menu
Order tab — cart and checkout flow	BUILT	Cart works — Square payment is STUB
Order tab — group order flow	ENTRY POINT	Coming Soon sheet — full flow is Phase 2
Rewards — points display and rewards	BUILT	Stub data — needs Square Loyalty API
Rewards — QR code for counter	PLACEHOLDER	QR generation not wired yet
Catering — Erik's Addiction Box	BUILT	Form present — Square payment STUB
Catering — Office Breakfast (\$16/person)	BUILT	Locked pricing in code
Catering — Gallon Cold Brew	COMING SOON	\$45/gallon locked — Square entry pending
Catering — Private Events form	BUILT	EventRequestSheet complete — backend STUB
Sunday tab — current lineup + vibe note	BUILT	Stub data — needs backend
Sunday tab — Play Request form	BUILT	Form complete — backend STUB
Account tab — profile and preferences	BUILT	Stub data — needs auth backend
Splash screen with real logo	BUILT	Fades in 0.6s, dismisses at 1.4s
ADMIN FEATURES		

Admin passphrase gate	BUILT	Set AdminSecrets.swift before distribution
Today Content Manager	BUILT	Changes don't persist without backend
Sunday Lineup Manager	BUILT	Slot management built — no persistence
Events Queue	BUILT	Displays submitted requests — no persistence
Customer Lookup	BUILT	Stub records — needs Square Customers API
Broadcast Push Composer	BUILT	UI complete — push sending needs FCM
Cold Brew Availability Manager	BUILT	Toggles built — no persistence
Weekly Recurring Calendar	BUILT	UI complete — no backend orders yet
INFRASTRUCTURE		
Square Catalog (menu built in Square)	NOT BUILT	Must be built before Order tab goes live
Square Loyalty activation	UNKNOWN	Needs verification on production account
Square payment processing	STUB	placeOrder() mocked — SDK not linked
Square sandbox verification	NOT STARTED	Phase 1 — 2-4 developer days
Hideout backend (API server)	NOT BUILT	Phase 2 — all live data depends on this
Push notifications (FCM/APNs)	NOT WIRED	Token registration exists, sending does not
Universal Links (NFC web fallback)	NOT BUILT	Needs apple-app-site-association on site
Real food photos	PENDING	10 photos needed — asset names in checklist
TestFlight / App Store submission	NOT STARTED	Phase 4 after core is stable

Photography Doctrine

Every photo in the Hideout app should make someone want the item and the patio at the same time.

The app currently shows warm gradient placeholders. They maintain the thermal palette and card structure. Real photography from the cafe is the single highest-impact change remaining before the app looks like a product rather than a prototype.

The photo philosophy is not about looking premium. It is about making ordering obvious. When someone sees the Erik's Addiction — the swirled green and brown cup with the Hideout sign behind it — they know exactly what they're ordering. The gradient placeholder does not do that.

Real photos only	Taken at Hideout, of actual Hideout food, in actual Hideout light. Customers who have been here will know.
No stock photography	Stock food photography breaks the identity contract. Customers who have ordered the Egg + Avocado Toast will know it doesn't look like a stock image.
Plastic cups stay	Erik's Addiction is served in a plastic cup. The cup is in the photo. Authenticity over idealization.
The patio matters	Photos on the patio or inside the cafe carry the ambient warmth of the space. The food and the place are one thing.
At-size readability	Thumbnails render at 76×76px. Close crop on the hero ingredient, not a wide environmental shot.
No AI-generated food	Never. This is non-negotiable.

PHOTOS NEEDED — IN PRIORITY ORDER

Asset Name	Item	Notes
photo_eriks_addiction	Erik's Addiction	Hero shot — also used as 16:9 Today card
photo_egg_avocado	Egg + Avocado Toast	Most ordered toast — catering anchor
photo_chicken_toast	Roasted Chicken Toast	Catering product
photo_cold_brew	Cold Brew	Also used for Gallon Cold Brew

photo_matcha_latte	Matcha Latte	Quick order chip
photo_breechay_special	Iced Breechay's Special	Visually distinctive swirl
photo_salmon_toast	Wild Caught Salmon Toast	Consumer advisory item
photo_pb_bowl	Brice's PB Bowl	Bowl format
photo_acai_bowl	Açaí Bowl	Bowl with toppings
photo_watermelon_juice	Watermelon Juice	Clean red visual

Drop photos into HideoutApp/Assets.xcassets using the asset names above. No code changes needed.

Roadmap

PHASE 0 — BEFORE SUNDAY (This Week)

No code required.

Set admin passphrase

Open AdminSecrets.swift, set a passphrase. Default is not secure for distribution.

Install on your phone

TestFlight or direct install. Real use reveals friction the simulator misses.

NFC stickers

Order NTAG213 stickers (\$8/10 on Amazon). Write URL: hideoutmiami.com/app?source=nfc. Stick one near the Square terminal.

5 product photos

Erik's Addiction, Egg + Avocado Toast, Roasted Chicken Toast, Cold Brew, Matcha Latte.

Test with 3-5 real people

Jimmy if willing. One office-adjacent contact. Watch what confuses them.

PHASE 1 — SQUARE SETUP (After Sunday)

One developer, 3-4 days. Proves API assumptions before backend is built.

- Square seller account confirmed in production
- Square Developer account + sandbox application created
- Square Catalog built in sandbox from the locked menu (17 items, correct modifiers)
- Square Loyalty activated and tested in sandbox
- Off-session auto-charge verified (Jimmy path — if this fails, recurring B2B model changes)
- Square Loyalty webhook vs polling decision (changes notification architecture)
- Square Catalog channel visibility verified (catering-only items hidden from POS)
- Square In-App Payments SDK linked — first end-to-end payment confirmed

PHASE 2 — BACKEND BUILD

After Phase 1 verification. The largest work block.

- Hideout API server (Node.js or Python, Railway or AWS)
- PostgreSQL database with all 16 data models from Build Decision Addendum
- Square webhook receiver with idempotency guarantees
- Live data wiring: menu, open/closed status, Sunday event, loyalty points
- B2B recurring order scheduler with auto-create and holiday handling
- Push notification infrastructure (FCM + APNs)
- Admin dashboard persistence

PHASE 3 — PHASE 2 FEATURES

- Group order flow — initiator → share link → members add items → one payment
- Gallon cold brew ordering — when Square catalog entry is confirmed
- Village steward mode — passphrase-gated Sunday attendance intake
- Space rental system — standardized booking flow from inquiry to confirmed event
- B2B recurring account self-registration — 5-step onboarding flow

PHASE 4 — APP STORE LAUNCH

- Apple Developer Program membership (\$99/year)
- TestFlight beta with 5-10 regulars
- App Store screenshots and description in Hideout brand voice
- Google Play — iOS first, Android 6 months after launch

The Governing Question

Does this make repeat interactions shorter?

This is the test for every future feature decision at Hideout.

Not: 'Is this useful?' — useful is too broad. Every feature is useful to someone. Not: 'Is this impressive?' — impressive is the wrong goal. Not: 'Does this add value?' — adding value is not the constraint. The constraint is repetition.

The app's power is not in what it does the first time. It is in what it does the tenth, fiftieth, two hundredth time. A regular who reorders their Erik's Addiction in 4 seconds on a Wednesday morning is the success state — not a customer who downloaded the app and explored all six tabs.

HOW TO USE THIS QUESTION

Scenario: Someone proposes adding a social feed to the Sunday tab

Does this make Sunday regulars' repeat interactions shorter? No — it adds content to scroll. It turns the room into content. It violates Village doctrine. It does not ship.

Scenario: Someone proposes a loyalty tier system (Bronze/Gold/Platinum)

Does this make repeat orders shorter? No — it adds a status layer the customer has to learn. It turns acknowledgment into a game. It does not ship.

Scenario: Someone proposes pre-filling Jimmy's standing order automatically

Does this make Jimmy's repeat interaction shorter? Yes — from 10 seconds to zero for routine orders. It ships.

Scenario: Someone proposes a 're-engagement' push after 2 weeks of inactivity

Does this make repeat interactions shorter? No — it creates an interaction where there was none. It manufactures engagement. It does not ship.

Scenario: Someone proposes showing the Express Order sheet when a regular taps the NFC sticker

Does this make repeat interactions shorter? Yes — their usual is on screen in 0.3 seconds. One tap. Done. It ships.

Six months from now, when you and Brice disagree about a feature, you will not ask: 'What do we feel like building?'

You will ask: 'Does this make repeat interactions shorter?'

That is when doctrine becomes useful. Not as a document — as a decision.

Design the conditions. Protect the standard. Remove the friction. Trust the people.

HIDEOUT MIAMI · THE VILLAGE · THE ROOM